

IBM Bob 2.0 Hackathon — Your Playbook

A do-this-then-this guide for building “Orient,” a smart developer-onboarding assistant.

Team minahiru · Solo entry · lablab.ai IBM Bob 2.0 Hackathon · 24 September 2026

Deadline — fill this in. This page didn't list the submission date; it's on the lablab event page. Write it here and everything else hangs off it:

Submission deadline: _____ Days left: _____

1. The whole thing in six sentences

You are building a tool called **Orient** that reads an unfamiliar codebase and produces an onboarding pack — the setup steps, a map of the code, and a list of where the old docs are wrong — so a new developer is productive in minutes instead of days.

You demonstrate it on a small, deliberately messy sample project. You prove it worked by having Bob complete a starter coding task using only the pack it generated. You submit a GitHub repo, a short video, and an impact write-up. That is the entire hackathon. It is a demo with a story, not a finished product.

What is “the repo”? This is the thing you were unsure about. Your repo is one GitHub folder holding three things: (1) the messy sample project Bob works on, (2) the Bob instruction files and the onboarding pack Bob produces, and (3) a `bob_sessions` folder of screenshots proving you used Bob. That's it. The next section shows the exact layout.

2. What the repo looks like

This is your target. Don't build it by hand — most of it gets generated. This is just so you can picture the finish line.

```

orient-hackathon/
├─ sample-app/           ← the messy project Bob investigates (I give you this)
│  ├─ src/               broken-ish Node.js + Express app
│  ├─ README.md          deliberately out-of-date, to be caught
│  ├─ .env.example        incomplete on purpose
│  └─ package.json
├─ .bob/                 ← your Bob instructions (I give you these)
│  └─ skills/
│     ├─ env-archaeologist.md
│     └─ doc-reconciler.md
├─ onboarding-pack/      ← what Bob PRODUCES when you run it
│  ├─ SETUP.md           the corrected setup steps
│  ├─ ARCHITECTURE.md    the map of the code
│  └─ DOC-CORRECTIONS.md “the README is wrong about these 5 things”
├─ bob_sessions/         ← REQUIRED screenshots of Bob task summaries
│  ├─ minahiru_task01_env_scan_summary.png
│  └─ minahiru_task02_doc_check_summary.png
└─ README.md             ← the top-level pitch (problem, solution, impact)

```

Who builds what. When I (Claude) write a file it costs zero Bobcoins. So I hand you `sample-app/` and `.bob/skills/` ready-made. Bob only spends coins on the runs that create `onboarding-pack/` — the part that must be genuine Bob work for your evidence.

3. The Bobcoin rule that decides everything

You get **40 Bobcoins total**. Every Bob action spends some. At 100% used, you get no more. Treat them like cash.

- **Never explore casually in Bob.** Think first, then run one focused task.
- **Do the free work off Bob:** designing, writing the sample repo, drafting the pitch — none of that needs Bob.
- **Sign into the hackathon account** (`ibm-coding-challenge-uat` , region us-east), never a personal Bob account, or you burn your own coins.
- **Budget roughly:** ~5 coins learning, ~25 building, keep ~10 in reserve for re-runs and the video.

4. Phase 0 — Set up (do this before anything else)

Goal: log into Bob successfully. Build nothing yet.

1. Register for the hackathon on lablab if you haven't. Your team name there is **minahiru**.
2. Check your email for an invite from the IBM Bob team ("added as a team member to ibm-hackathon-..."). Check spam. This is your provisioned account, not a real team.
3. Create an **IBMid** on the same email if you don't have one.
4. Install **Bob IDE v2.0.2 or later** (older versions stop working Sept 30). Sign in.
5. In Bob → Settings → General, confirm the active instance is `ibm-coding-challenge-uat (us-east)` and that you can see your Bobcoin balance.

Do not continue until step 5 works. Every year people build for hours on the wrong account and waste their own coins. Confirm the account first.

5. Phase 1 — Get the sample project

Goal: have the messy `sample-app/` on your computer. Cost: 0 coins.

1. I hand you the `sample-app/` folder and the two skill files (ask me next — "build the repo").
2. Make a new GitHub repository called `orient-hackathon`.
3. Put the folder in it. Open the folder in Bob IDE.

Why a made-up project and not a real one? Because you control the pain points, it's safe (no client or personal data, which the rules forbid), and judges can reproduce it. This is a strength, not a shortcut — say so in your write-up.

6. Phase 2 — Learn Bob with one tutorial

Goal: know the buttons before you spend coins on your own build.

In the hackathon guide's exercise list, do "**Inspect an unfamiliar codebase.**" It teaches the exact moves your project needs: pointing Bob at code, using Ask mode to understand it, using Agent mode to save results. It costs a few coins and it is the best few coins you'll spend. Also skim "**Skills**" and "**Subagents**" in the guide so you recognise those features when you use them.

7. Phase 3 — Build Orient (the core)

Goal: Bob generates the onboarding pack. This is the heart of your submission.

Open `sample-app/` in Bob IDE, on the hackathon account. Run these as separate tasks. **After each one, take the screenshot** (Phase 5) before moving on.

Task 1 — The environment archaeologist

Load the skill I gave you, then in Bob's chat paste:

```
Using the env-archaeologist skill, scan this project for every
environment variable the code actually reads. For each one give the
name, the file and line, whether it's required, and any default.
Then produce a complete .env.example and a SETUP.md with the real
steps to run this app on a clean machine. Save both to
../onboarding-pack/. Do not invent variables; mark anything unclear
as UNKNOWN.
```

Task 2 — The doc reconciler

```
Using the doc-reconciler skill, compare the claims in README.md
against what the code actually does. List every place the README is
wrong, outdated, or missing – with the file and line that proves
it. Save the result as ../onboarding-pack/DOC-CORRECTIONS.md.
```

Task 3 (optional, if coins allow) — The map

```
Analyse this codebase and produce ARCHITECTURE.md: the entry point,
the main modules and what each does, the request flow, and any dead
or unused code. Use a Mermaid diagram for the structure. Save to
../onboarding-pack/.
```

Use the right mode. Ask mode to investigate (cheaper, no file changes), Agent mode when you tell it to save files. If Bob supports subagents for these, letting each skill run as its own subagent is exactly the “parallel tasks / subagents” the judges want to see — but only if coins allow. A clean sequential run beats a broken parallel one.

8. Phase 4 — Prove it worked

Goal: turn “looks good” into “it demonstrably works.” This wins.

Start a **fresh Bob conversation** (new context window, so it has no memory of the earlier work) and give it only the onboarding pack:

Read onboarding-pack/SETUP.md and get this app running. Then complete this starter task: [a small change with a test that passes or fails, e.g. “make GET /health return the app version”]. Confirm the app starts and the change works.

If a fresh agent can set up and change the app using only the pack, that is your proof. Capture it in the video.

Your impact numbers — measure both sides

METRIC	MANUAL (YOU, FIRST)	WITH ORIENT
Minutes to a running app	time yourself	time it
README claims that were wrong	—	count from the pack
Env vars missing from .env.example	—	count from the pack

Before you let Bob touch it, try setting the app up yourself with a stopwatch. That manual number is your before. Be honest — “one run, one person” is fine to state.

9. Phase 5 — The required evidence (don't skip)

Mandatory: screenshots of Bob task summaries, or the entry may not be judged.

1. In Bob IDE chat, open the **Tasks** panel.
2. Click a task related to your project. Click the **task header** — a session consumption summary appears.
3. Screenshot it. Save as PNG, named like `minahiru_task01_env_scan_summary.png`.
4. Repeat for every real task. Put them all in the `bob_sessions/` folder in your repo.

Do this as you go, not at the end. Take the screenshot right after each task in Phase 3 and 4. If you wait, you'll forget which task was which.

10. Phase 6 — Package and submit

Goal: three deliverables, then hit submit.

1. **Top-level README.md** — the pitch. Structure: the problem (onboarding wastes days), what Orient does, the impact table, how to reproduce, and honest limitations. I can write this for you.
2. **Demo video, 2–3 min** — narrate your screen: the problem, run a task, show the generated pack, show the fresh agent using it, end on the numbers. Rough and clear beats slick and vague.
3. **Push everything to GitHub** — sample-app, .bob, onboarding-pack, bob_sessions, README. Confirm `bob_sessions/` is really there.
4. **Fill the lablab submission form** — title, description, repo link, video link. Submit before the deadline; don't wait for the last hour.

11. One-page checklist

- ☐ Registered on lablab as team minahiru
- ☐ Bob hackathon account confirmed; instance is ibm-coding-challenge-uat; coins visible
- ☐ Bob IDE v2.0.2+ installed and signed in
- ☐ Sample-app + skills from Claude, in a GitHub repo, opened in Bob
- ☐ Did the “Inspect an unfamiliar codebase” tutorial
- ☐ Timed the manual setup myself (the “before” number)
- ☐ Task 1 env scan run + screenshot saved
- ☐ Task 2 doc check run + screenshot saved
- ☐ Onboarding pack generated (SETUP, DOC-CORRECTIONS, +/- ARCHITECTURE)
- ☐ Verification: fresh agent set up the app + did the starter task
- ☐ Impact numbers filled in
- ☐ bob_sessions folder has all screenshots

- ☐ Top-level README pitch written
- ☐ Demo video recorded
- ☐ Repo pushed; lablab form submitted

If you only have time for the minimum: Phases 0, 1, 3 (Task 1 + Task 2 only), 5, and 6. A messy repo, two Bob skills that produce a real onboarding pack, the screenshots, and a 3-minute video is a complete, competitive entry. Everything else is polish.

Next step: tell Claude “build the repo” and you’ll get the sample-app and the two skill files ready to drop into GitHub — at zero Bobcoin cost. And send the deadline so the plan can be dated.